

COMPREHENSIVE AUDIT REPORT

Security & Disaster Recovery Audit

A thorough evaluation of the AI Reef AI Madeena Real Estate Dashboard's security posture, data protection mechanisms, and disaster recovery readiness against industry best practices and UAE regulatory requirements.

AI Reef AI Madeena
Real Estate Dashboard
June 9, 2026

CONFIDENTIAL

Table of Contents

1. Executive Summary	3
2. Authentication and Authorization Security	4
2.1 NextAuth Configuration	4
2.2 CRITICAL: Missing Middleware	4
2.3 CRITICAL: Hardcoded Default Passwords	4
2.4 Role-Based Access Control Gaps	5
3. API Route Protection Analysis	5
3.1 Unprotected Routes Requiring Immediate Action	5
3.2 Rate Limiting Deficiencies	6
3.3 Input Sanitization Gap	6
4. Data Protection and Privacy	6
4.1 Encryption at Rest and in Transit	6
4.2 Multi-Tenant Data Isolation	6
4.3 Password Policy Inconsistency	7
4.4 CRON_SECRET Exposure	7
5. Disaster Recovery and Backup Assessment	7
5.1 Current Backup Capabilities	7

5.2 CRITICAL: Production Cutover Destroys Backup Records	8
5.3 7-Day Recovery Assessment	8
5.4 Restore Mechanism Limitations	9
6. Environment Variables and Secrets Management	9
7. Prioritized Remediation Plan	10
7.1 P0 - Critical (Immediate Action Required)	10
7.2 P1 - High Priority (Next Sprint)	11
7.3 P2 - Medium Priority (Roadmap)	12
7.4 P3 - Lower Priority (Future Enhancement)	12
8. Disaster Recovery Readiness Checklist	13

1. Executive Summary

This document presents a comprehensive security and disaster recovery audit of the Al Reef Al Madeena Real Estate Dashboard, a production SaaS application serving the UAE real estate market. The audit was conducted against industry best practices for web application security, data protection regulations, and business continuity standards. The application is built on Next.js with TypeScript, Prisma ORM, NextAuth v5, and deployed on Vercel with a Neon PostgreSQL database.

The audit reveals a **mixed security posture**. While the application has strong foundational elements such as JWT-based authentication with brute-force lockout, role-based access control on most API routes, soft-delete data protection, and daily automated backups with SHA-256 integrity hashing, there are several critical gaps that could expose client data to unauthorized access or irreversible loss. The most severe findings include the complete absence of middleware-level route protection, hardcoded default passwords in the setup endpoint, a health endpoint leaking cross-tenant data without authentication, and a production cutover endpoint that destroys all backup records upon execution.

On the disaster recovery front, 7-day data recovery is achievable but only under specific conditions: the Vercel Blob storage token must be properly configured, Neon PITR must be enabled at the provider level, and the production cutover endpoint must never be executed without a pre-cutover backup. Currently, five entity types (Reservations, RentAdjustments, Receipts, Notifications, ScoreAuditLogs) are not included in backup or restore operations, which means partial data loss is possible even after a successful restore.

Severity	Count	Key Issues
CRITICAL	2	No middleware; hardcoded default passwords in setup
HIGH	4	Health endpoint data leak; missing RBAC on adjustments; staff can modify reservation finances; no rate limit on 2FA validate
MEDIUM	6	Weak password policy in self-reset; unused sanitization; receipt XSS; empty CRON_SECRET; token email leak; cutover auth bypass
LOW	4	No CSRF tokens; no request size limits; error message leakage; TS build errors ignored

Table 1: Vulnerability Severity Distribution

2. Authentication and Authorization Security

2.1 NextAuth Configuration

The application uses NextAuth v5 with a Credentials-only provider and JWT session strategy. Sessions have an 8-hour maximum age, which is reasonable for a business application. The system implements a database-backed brute-force lockout mechanism: after 5 failed login attempts, the account is locked for 15 minutes. This lockout state persists in the database, meaning it survives server restarts and cannot be bypassed by clearing browser cookies. Audit logging is implemented for all login events, and soft-deleted users are correctly blocked from authenticating.

However, there are concerns regarding the NEXTAUTH_SECRET environment variable. The .env file in the repository only contains the DATABASE_URL, and the secret is expected to be provided through the hosting environment. If this variable is not set in production, JWT signing falls back to an unsafe default or fails entirely. The application does validate this at startup through the env.ts configuration module, but the incomplete .env file creates confusion and risk during local development or redeployment scenarios. Additionally, the Credentials-only provider means there is no multi-factor authentication at the provider level, although the application does implement its own TOTP-based 2FA system.

2.2 CRITICAL: Missing Middleware

The most significant architectural security gap is the complete absence of a middleware.ts file. In Next.js, middleware runs before any API route handler or page component executes, making it the ideal location for enforcing authentication at the route level. Without middleware, every single API route must individually call `getAuthUser()` to verify the session. If any developer forgets this call, the route becomes fully accessible to unauthenticated users with no safety net.

This is not a theoretical concern. The audit found that the `/api/health` endpoint exposes sensitive data including cross-tenant record counts and environment variable configuration status without any authentication check. This is a direct consequence of the missing middleware layer. A proper middleware should protect all routes except an explicit allowlist (`/api/auth/*`, `/api/setup` POST, `/api/health` basic liveness, `/api/reset-requests` POST, `/api/auth/forgot-password` POST) and redirect unauthenticated page requests to the login page while returning 401 for unauthenticated API requests.

2.3 CRITICAL: Hardcoded Default Passwords

The `/api/setup` endpoint, which creates initial admin, owner, and staff users, contains hardcoded default passwords as fallbacks when the request body does not include explicit passwords. Specifically, the code uses "admin2024" for the admin user, "owner123" for the owner, and "staff123" for staff. While the endpoint is guarded by a production check that prevents execution in production environments, any staging, development, or testing environment is at risk. These passwords are extremely weak and do not

even meet the application's own password policy of 8 characters with at least one uppercase letter and one number. The default passwords must be removed entirely, and the endpoint should require explicit passwords that meet the policy.

2.4 Role-Based Access Control Gaps

The application implements a four-tier role system (admin, owner, manager, staff) with most API routes properly enforcing role-based restrictions. However, the audit identified specific gaps where staff-level users can perform financial operations that should be restricted. The `/api/adjustments/[id]` PUT and DELETE endpoints check authentication but do not enforce any role-based access control, meaning any authenticated user including staff can edit or cancel rent adjustments, which are financial records. Similarly, the `/api/reservations/[id]` PUT endpoint allows staff to modify financial fields such as `depositAmount`, `depositAppliedTo`, and `depositAppliedAmount`, which should be restricted to owner and admin roles only. These gaps could allow unauthorized financial manipulation by lower-privileged users.

3. API Route Protection Analysis

A thorough review of all API routes was conducted to determine their authentication and authorization status. The application has 37 distinct API route handlers. Of these, 27 are properly protected with authentication checks and appropriate role-based access control. The remaining 10 routes are intentionally or unintentionally unprotected. While some unprotected routes are necessary (such as login and password reset endpoints), others represent genuine security vulnerabilities.

3.1 Unprotected Routes Requiring Immediate Action

Route	Method	Risk	Issue
/api/health	GET	HIGH	Exposes DB counts, env var status, backup info without auth
/api/setup	POST	CRITICAL	Creates users with hardcoded default passwords
/api/auth/2fa/validate	POST	HIGH	No rate limiting; TOTP brute-force possible
/api/cron/daily-report	GET	MEDIUM	CRON_SECRET optional; empty secret accepted
/api/backup/auto	GET	MEDIUM	CRON_SECRET passed as query parameter (logged in access logs)

Table 2: Unprotected Routes Requiring Immediate Remediation

3.2 Rate Limiting Deficiencies

The 2FA validation endpoint (`/api/auth/2fa/validate`) accepts TOTP codes without any rate limiting. TOTP codes are 6-digit numbers, giving an attacker 1,000,000 possible combinations. While codes rotate every 30 seconds, an attacker making even modest request rates could brute-force valid codes within the time window. The application has a robust brute-force lockout for the primary login endpoint, but this protection does not extend to the 2FA validation step, creating a significant gap in the authentication chain. A rate limit of 5 attempts per 5 minutes per email address with exponential backoff would substantially mitigate this risk.

3.3 Input Sanitization Gap

The application defines sanitization functions (`sanitizeString`, `sanitizeRichString`, `sanitizeEmail`) in the `api-utils.ts` module, but these functions are never called in any API route. All user-supplied string input, including tenant names, notes, descriptions, and company information, is stored in the database exactly as received without any sanitization. This creates a stored cross-site scripting (XSS) risk when this data is rendered in the user interface. Additionally, the receipt HTML generation endpoint (`/api/receipts/[id]/html`) interpolates user data directly into HTML via template literals without escaping, which means a malicious tenant name containing script tags would execute when the receipt is rendered in a browser.

4. Data Protection and Privacy

4.1 Encryption at Rest and in Transit

Data in transit is protected by HTTPS, which is enforced by Vercel's infrastructure. All connections between the client browser and the Vercel edge network, as well as between Vercel and the Neon PostgreSQL database, use TLS encryption. This meets the baseline requirement for protecting data during transmission. However, data at rest in the Neon PostgreSQL database is encrypted using Neon's default encryption, which is adequate but not configurable at the application level. Backup files stored in Vercel Blob use private access tokens but do not have an additional encryption layer. The backup JSON files contain all tenant data including names, phone numbers, addresses, and financial records, which represents personally identifiable information (PII) stored without application-level encryption. Adding AES encryption before blob upload would provide defense-in-depth for this sensitive data.

4.2 Multi-Tenant Data Isolation

The application uses a shared-database multi-tenant architecture with `companyId` as the isolation key. All API routes filter data by the authenticated user's `companyId`, which provides logical separation

between tenants. However, the `/api/health` endpoint exposes aggregate database counts across all companies, representing a cross-tenant data leakage vulnerability. While individual tenant records are not exposed, the aggregate counts reveal business intelligence such as the total number of tenants, properties, and payments across all companies using the platform. This information could be valuable to competitors or malicious actors. The health endpoint should either require authentication for detailed information or split into a public liveness check and an authenticated deep health check.

4.3 Password Policy Inconsistency

The application enforces different password policies depending on the context. The signup endpoint and admin-initiated password resets require passwords with at least 8 characters, one uppercase letter, and one number. However, the self-service password reset endpoint (`/api/auth/reset-password`) only enforces a minimum of 6 characters with no complexity requirements. This inconsistency means that users who reset their own passwords can set weaker passwords than the policy intends, undermining the overall password security posture. The self-service reset should enforce the same policy as the signup and admin reset endpoints to maintain consistent security standards across the application.

4.4 CRON_SECRET Exposure

The `CRON_SECRET` is hardcoded in the `vercel.json` configuration file, which is committed to the source code repository. This means anyone with read access to the repository can discover the secret and trigger cron-only endpoints such as daily report generation and automated backups. Furthermore, the cron endpoint accepts an empty or undefined `CRON_SECRET` value, falling back to checking the Vercel-specific `x-vercel-cron` header, which can be spoofed outside of Vercel's infrastructure. The `CRON_SECRET` should be moved to an environment variable only, the `vercel.json` should reference it dynamically, and the endpoints should reject requests when the secret is not configured. Additionally, the auto-backup endpoint passes the cron secret as a query parameter, which may be logged in server access logs, further exposing the secret.

5. Disaster Recovery and Backup Assessment

5.1 Current Backup Capabilities

The application implements a dual-track backup system consisting of manual on-demand backups and daily automated backups. Manual backups are triggered through the UI by owner or admin users and generate a JSON file containing Company, Properties, Tenants with payments, Expenses, Maintenance, Users (passwords excluded), and the last 1000 AuditLog entries. The file is downloadable to the browser with a SHA-256 integrity hash computed and stored in a BackupRecord in the database. Automated backups run daily at 22:00 UTC via a Vercel Cron job, cover all companies, and attempt to upload to Vercel Blob storage if the `BLOB_READ_WRITE_TOKEN` is configured. Auto-backups have a 90-day

retention policy that automatically deletes old backup records and their corresponding blob files.

The backup system has a significant coverage gap. Five entity types are not included in either manual or automated backups: Reservations, RentAdjustments, Receipts, Notifications, and ScoreAuditLogs. This means that even after a successful restore from backup, these records would be permanently lost. Reservations contain deposit amounts and prospect information, RentAdjustments affect tenant billing, and Receipts are financial documents that may have legal significance. The absence of these entities from the backup scope represents a material data loss risk for any business relying on the platform for complete financial record-keeping.

5.2 CRITICAL: Production Cutover Destroys Backup Records

The most dangerous gap in the disaster recovery posture is the `/api/production-cutover` endpoint. This endpoint is designed to remove all demo and sample data when transitioning from a demo environment to production. However, it hard-deletes ALL data including BackupRecord entries. Once backup records are deleted, the SHA-256 integrity hashes are lost, the metadata about when backups were taken is gone, and there is no trace of previous backups in the database. While the actual backup files in Vercel Blob storage may still exist, the application has no way to reference or restore them. This endpoint effectively eliminates the disaster recovery safety net at the exact moment when production data is most valuable.

The company reset endpoint (`/api/company/reset`) presents a similar but less severe risk. It soft-deletes properties, tenants, expenses, and maintenance records, but hard-deletes all payments and audit logs before soft-deleting tenants. Hard-deleting payments means they cannot be recovered through the soft-delete mechanism, and hard-deleting audit logs destroys the forensic trail of what happened. There is no pre-reset backup requirement, no undo mechanism, and the audit trail that could document the reset is destroyed in the process.

5.3 7-Day Recovery Assessment

Scenario	Recoverable?	Mechanism
Data corrupted within 7 days	Yes	Vercel Blob daily snapshots (90-day retention) + Neon PITR
Accidental deletion by user	Yes	Soft-deleted records remain in DB; manual backup download; auto-backup blobs
Full database loss	Partial	Requires BLOB_READ_WRITE_TOKEN configured and Neon PITR enabled. No documented DR runbook
Data entry error 5 days ago	Difficult	Can restore from 5-day-old backup, but restore is upsert-based (does not remove incorrect data)

Selective record recovery	Manual	Must download backup JSON, manually extract records, and import
Ransomware or malicious deletion	Unlikely	Auto-backups stored in same DB; production-cutover hard-deletes backup records

Table 3: 7-Day Data Recovery Feasibility Assessment

Verdict: 7-day recovery is achievable but conditional. It requires three preconditions to be met: (1) the BLOB_READ_WRITE_TOKEN must be configured so that cloud-stored backups survive database loss, (2) Neon PITR must be enabled at the provider level, and (3) the production-cutover endpoint must never be executed without a prior manual backup download. Without any one of these conditions, 7-day recovery is not guaranteed. Currently, there is no monitoring or alerting to verify that these conditions remain true over time, and no documented disaster recovery runbook to guide administrators through the recovery process in a high-stress scenario.

5.4 Restore Mechanism Limitations

The restore endpoint (/api/backup POST) uses an upsert-based approach rather than a destructive replace. This means that existing data is merged with backup data rather than wiped and replaced. While this is safer in some scenarios (it does not delete data that was not in the backup), it creates a significant limitation: incorrect or corrupted data that was entered after the backup was taken will persist alongside the restored data. There is no option for a clean-slate restore that wipes the database and then imports from backup. Additionally, there is no pre-restore automatic backup of the current state, meaning that if a restore goes wrong, the administrator has no way to revert to the pre-restore state. The SHA-256 hash computed during backup is stored in the BackupRecord but is never re-verified during restore, meaning a corrupted backup file could be imported without detection.

6. Environment Variables and Secrets Management

The application relies on several sensitive environment variables for security: NEXTAUTH_SECRET for JWT signing, DATABASE_URL for database connectivity, BLOB_READ_WRITE_TOKEN for backup storage, and CRON_SECRET for protecting automated endpoints. The audit identified multiple concerns regarding how these secrets are managed and protected throughout the application lifecycle.

Secret	Storage	Issue	Risk Level
NEXTAUTH_SECRET	Missing from .env	Expected from hosting env; fallback to unsafe default if not set	HIGH

CRON_SECRET	Hardcoded in vercel.json	Committed to source control; anyone with repo access can trigger cron endpoints	CRITICAL
BLOB_READ_WRITE_TOKEN	Environment variable	Optional; if not set, backups are DB-only records (lost if DB is lost)	HIGH
DATABASE_URL	In .env file	Committed to source control (should be in .gitignore)	MEDIUM

Table 4: Secrets Management Assessment

The most pressing concern is the CRON_SECRET being hardcoded in vercel.json. This file is part of the source code repository, meaning that anyone with read access to the repository (including all collaborators and anyone who can view the repository on GitHub) can discover the secret. The vercel.json file should use environment variable references rather than hardcoded values, and the actual secret should only exist in the Vercel project's environment variables. Additionally, the .env file in the repository contains the DATABASE_URL, which includes database credentials. While .env files should be in .gitignore, their presence in the repository structure suggests they may have been committed at some point.

7. Prioritized Remediation Plan

The following remediation plan is organized by priority level, with P0 items representing critical gaps that should be addressed immediately, P1 items representing high-priority improvements that should be implemented within the next sprint, and P2/P3 items representing medium and lower priority enhancements for the roadmap.

7.1 P0 - Critical (Immediate Action Required)

#	Issue	Remediation	Impact
1	No middleware - defense-in-depth absent	Create src/middleware.ts that enforces auth on all /api/* routes except explicit allowlist; redirect unauthenticated page requests to login	Prevents unauthenticated access to any route that forgets auth check
2	Hardcoded default passwords in /api/setup	Remove default password fallbacks; require explicit passwords that meet the 8+char + uppercase + number policy	Eliminates weak credential risk in non-production environments

3	CRON_SECRET hardcoded in vercel.json	Move to environment variable only; remove from source code; reject requests if CRON_SECRET is undefined	Prevents unauthorized cron endpoint access
4	Production cutover destroys backup records	Exclude BackupRecord from deletion in production-cutover; add pre-cutover mandatory backup step	Preserves DR safety net during transition to production
5	Ensure BLOB_READ_WRITE_TOKEN is configured	Verify token is set in Vercel environment; add startup warning if missing; add health check alerting	Ensures cloud persistence of backups survives DB loss

Table 5: P0 Critical Remediation Items

7.2 P1 - High Priority (Next Sprint)

#	Issue	Remediation	Impact
6	Health endpoint leaks cross-tenant data	Split into public liveness check (simple OK) and authenticated deep health check	Prevents information disclosure to unauthenticated users
7	Missing RBAC on adjustments PUT/DELETE	Add isFinancialUser() check to both handlers	Restricts financial record modification to authorized roles
8	No rate limiting on 2FA validate	Add per-email rate limit (5 attempts / 5 min) with exponential backoff	Prevents TOTP brute-force attacks
9	Staff can modify reservation financial fields	Restrict depositAmount, depositAppliedTo, depositAppliedAmount to owner/admin only	Prevents unauthorized financial manipulation
10	Incomplete backup coverage	Add Reservations, RentAdjustments, Receipts, Notifications, ScoreAuditLogs to backup/restore	Ensures complete data recovery capability
11	Unused sanitization functions	Apply sanitizeString() to all user-supplied string input before DB storage	Prevents stored XSS across the application
12	Verify SHA-256 hash on restore	Compare stored hash with file hash before importing; reject if mismatch	Prevents corrupted backup from being imported

Table 6: P1 High Priority Remediation Items

7.3 P2 - Medium Priority (Roadmap)

- Add pre-restore automatic backup of current state before any restore operation begins, ensuring a rollback path exists if the restore produces unexpected results.
- Change company reset to soft-delete payments instead of hard-deleting them, preserving the ability to recover payments through the soft-delete mechanism if the reset was performed in error.
- Create a documented Disaster Recovery runbook with step-by-step recovery procedures for the most common scenarios: database corruption, accidental deletion, full database loss, and ransomware.
- Add backup failure alerting through push notifications or webhook integrations, so administrators are immediately informed when automated backups fail rather than discovering the failure days later.
- Implement periodic automated integrity checks that compare current data hashes with the last backup hash to detect data corruption proactively before it becomes a recovery scenario.
- Add request body size limits (1MB for most endpoints) to prevent memory exhaustion attacks through extremely large JSON payloads.
- Return generic error messages to clients while logging detailed errors server-side only, preventing internal system details from being exposed through error responses.

7.4 P3 - Lower Priority (Future Enhancement)

- Unify backup schema versions (v1.0 manual vs v1.1 auto) to prevent restore incompatibility issues when backup formats diverge.
- Add cross-region backup replication by replicating the Neon database branch or Vercel Blob to a secondary geographic region for geo-redundancy.
- Implement AES encryption for backup JSON files before Vercel Blob upload, providing defense-in-depth for PII stored in cloud backup files.
- Add CSRF token protection for session-based mutations, especially for destructive operations such as company reset, user deletion, and payment deletion.
- Remove the typescript.ignoreBuildErrors flag in next.config.ts and fix underlying type errors, as suppressed type errors could mask security-relevant type confusion vulnerabilities.
- Remove the reset token email enumeration in /api/auth/verify-reset-token to prevent an attacker who obtains a reset token from discovering the associated email address.

8. Disaster Recovery Readiness Checklist

The following checklist summarizes the current state of disaster recovery readiness and the target state after implementing the recommended remediations. This can be used as a tracking tool for the development team to verify that all necessary protections are in place before the application handles live production data for paying customers.

Capability	Current State	Target State	Status
Daily automated backups	Yes (90-day retention)	Yes with alerting and monitoring	Partial
Cloud backup storage	Conditional (BLOB token required)	Always enabled with encryption	Gap
Complete data coverage	5 entity types excluded	All 15 entity types included	Gap
7-day point-in-time recovery	Daily snapshots only (no intra-day)	PITR via Neon + daily snapshots	Gap
Backup integrity verification	SHA-256 computed but not verified on restore	Hash verified before restore	Partial
Pre-restore safety backup	Not implemented	Automatic before every restore	Gap
Destructive restore mode	Upsert only (cannot remove bad data)	Wipe-then-restore option with double confirmation	Gap
DR runbook documented	Not documented	Step-by-step procedures for all scenarios	Gap
Backup monitoring and alerting	Health check flags at 48h	Push notification + webhook on failure	Partial
Production cutover safety	Destroys backup records	Preserves backup records; mandatory pre-cutover backup	Gap
Cross-region redundancy	Single region only	Secondary region replication	Gap

Table 7: Disaster Recovery Readiness Checklist